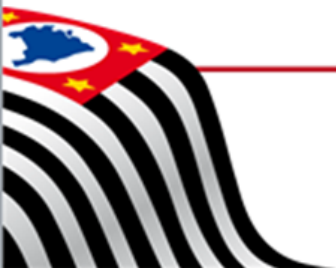


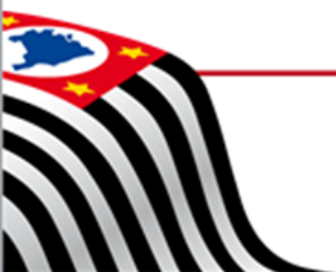
Programação Orientada a Objetos POO

(Object-Oriented Programming)
OOP



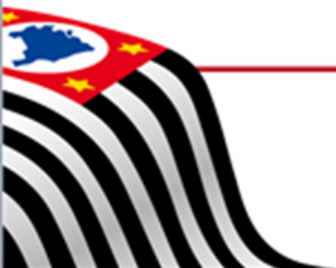
Objetivos

- Origem da POO
- Introdução à POO
- Visão geral da POO.
- Conceito de objeto, classe, atributo e método



Programação estruturada

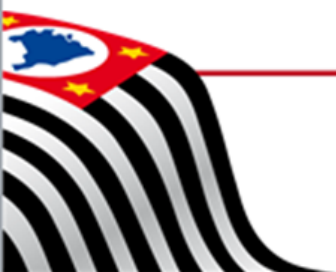
- Conjunto de programas
- Atuam sobre conjunto de dados
- Subprogramas
- Obter resultados



Programação estruturada

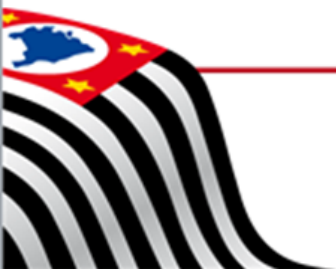
Grandes programas (muitas linhas)

- Pequenas modificações: afetam mudanças em vários programas.
- Custo da recompilação



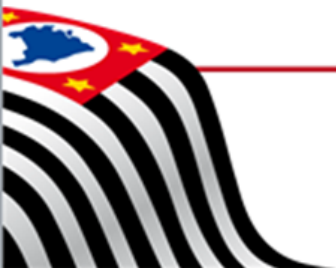
Origem da P.O.O.

- Início da década de 1960
- Necessidade de simulações em computadores.
- Reaproveitamento de códigos



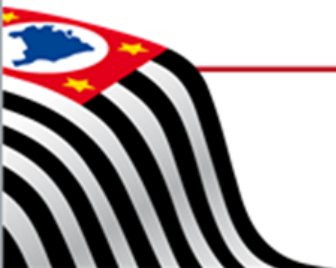
SIMULA 67

- Primeira linguagem de programação para simulação em computadores.
- Pesquisadores noruegueses:
Ole-Johan Dahl e Kristen Nygaard



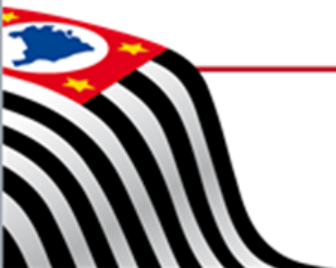
SmallTalk

- Década de 1970
- Desenvolvida por Alan Kay da empresa XEROX.
- Popularizou e incentivou o uso da POO.



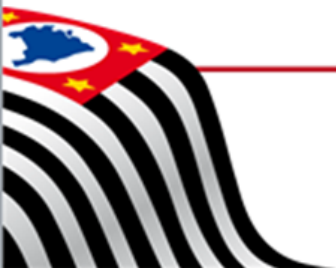
Características de uma Linguagem Orientada a Objetos

- Tipos de dados abstratos
- Herança
- Vinculação dinâmica



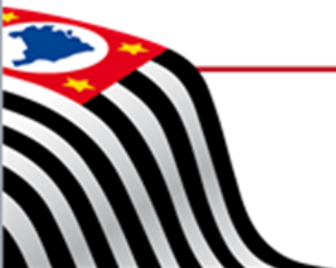
Modelagem de sistemas por objetos

Enxerga o mundo como um conjunto de objetos que interagem entre si e apresentam características e comportamentos próprios representados por seus atributos e seus métodos.



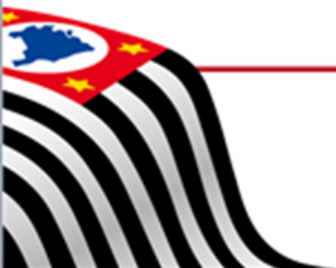
Modelagem de sistemas por objetos

- Objeto
- Classe
- Atributo
- Método
- Herança
- Encapsulamento



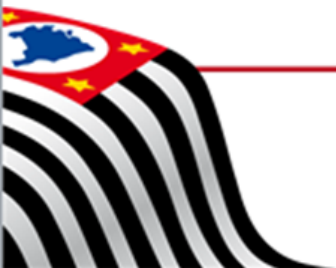
Conceito de Abstração

Representação de uma entidade que inclui somente os atributos de importância em um contexto particular.



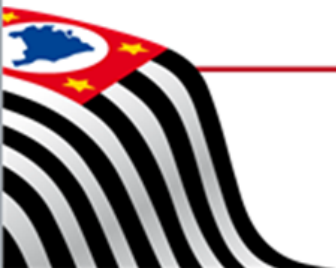
Encapsulamento

Empacotamento de atributos e operações
em objetos.



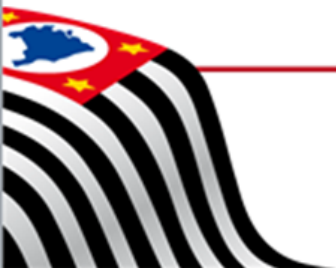
Herança

Uma forma de reutilização de software em que uma nova classe é criada absorvendo membros de uma classe existente com capacidades novas ou modificadas.



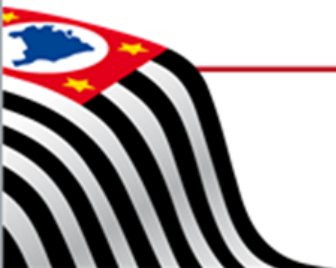
Herança

- Economia de tempo
- Aumenta a probabilidade de implementação e manutenção de um sistema.



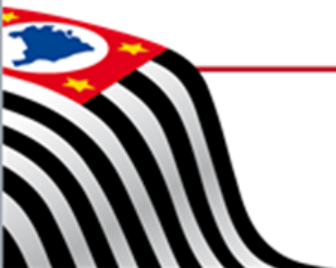
Vantagens da P.O.O.

- Manutenção e extensão do código
- Reaproveitamento do código
- Gerenciamento do desenvolvimento com grandes equipes. Divisão de classes e pacotes.



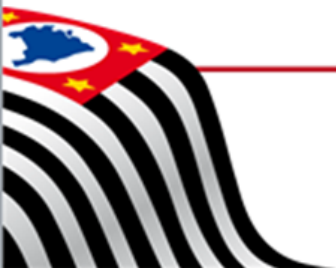
Linguagens Orientadas a Objetos

- Smalltalk
- C#
- C++
- VB.NET
- Ruby
- Java



Classe

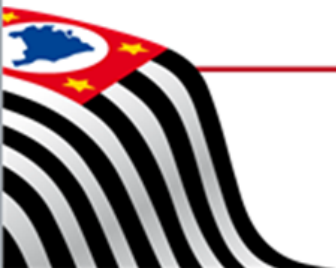
É um projeto que define objetos descrevendo seus atributos e métodos.



Classe

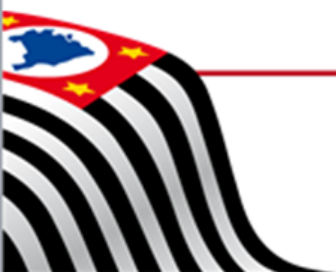
Exemplos

- Planta de uma casa
- Projeto de um automóvel



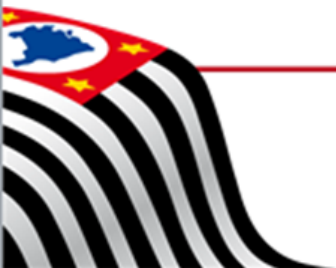
O que é um objeto

Objeto é uma extensão do conceito de objeto do mundo real.



Exemplo

- Pessoa
- Casa
- Automóvel
- Computador
- Relatório

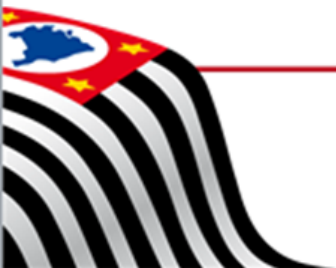


Atributos

- Características do objeto.

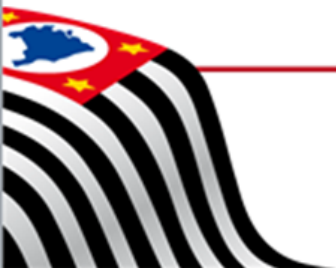
Exemplo:

Um objeto Carro possui:
cor, número de portas, marca, modelo



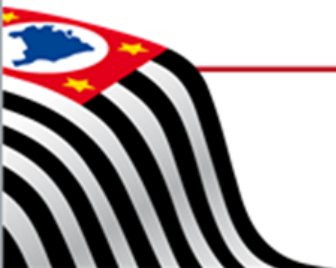
Atributos

- São representados como variáveis em uma declaração de classe.
- As variáveis são chamadas campos e estão declaradas na classe, mas fora do escopo das declarações de método da classe.



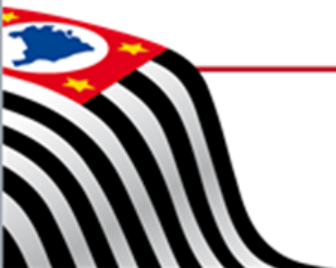
Variável de instância

- Quando cada objeto de uma classe mantém sua própria cópia de um atributo, o campo que representa o atributo também é conhecido como uma **variável de instância**



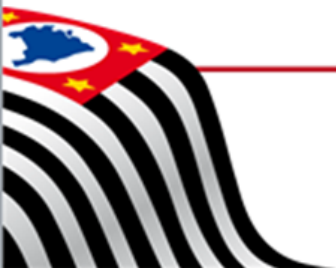
Criação de instância de classe

<nome classe> <nome objeto> = new <nome classe>



Palavra chave new

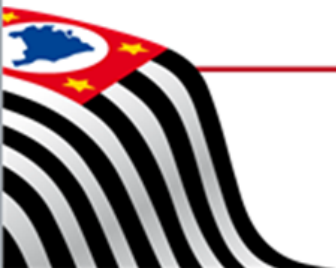
Cria um novo objeto da classe especificada.



Exemplo

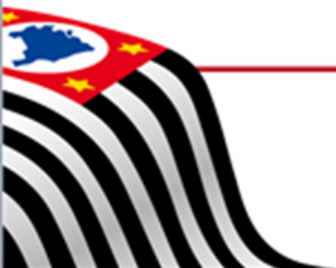
Classe Pessoa com os atributos nome, email e idade.

Criar uma instância da classe Pessoa, atribuir os valores e imprimir.



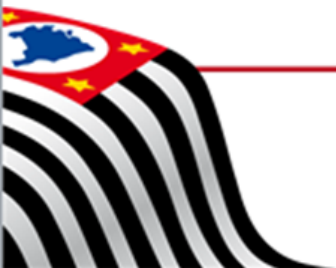
Exemplo

```
public class Pessoa {  
  
    public String nome, email;  
    public int idade;  
  
}
```



Exemplo

```
public class Main {  
    public static void main (String args [ ] ){  
        Pessoa X = new Pessoa ( );  
        X.nome = "João";  
        X.email = "joao@gmail.com";  
        X.idade = 27;  
        System.out.println ("Nome" + X.nome);  
        System.out.println ("E-mail" + X.email);  
        System.out.println ("Idade " + X.idade);  
    }  
}
```

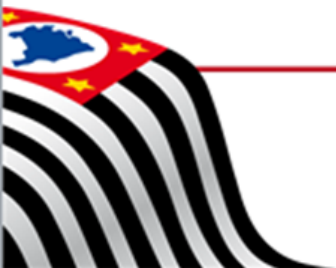


Métodos

- Operações que o objeto executa.

Exemplo:

Um objeto Carro possui operações:
dar partida, acelerar, frear, estacionar



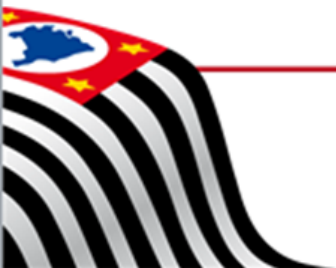
Métodos

Enviar mensagens para um objeto

Chamada de método

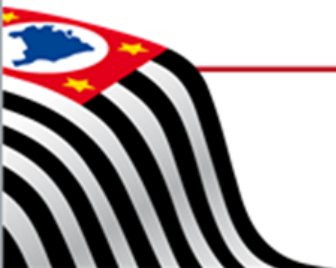
Exemplo:

Um objeto Carro precisa ser dirigido:
dar partida, acelerar, frear, estacionar



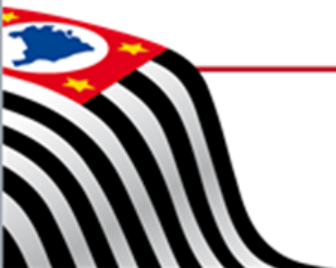
Modificador de visibilidade

- public
- private
- protected



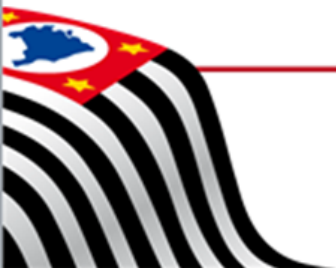
Modificador de visibilidade

- **public:** (público) o atributo ou método de um objeto dessa classe pode ser acessado por qualquer outro objeto.



Modificador de visibilidade

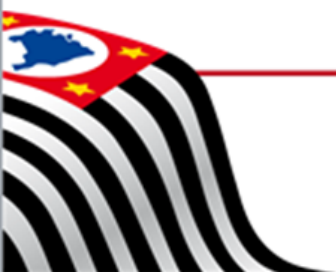
- **private:** (privado) o atributo ou método de um objeto dessa classe não pode ser acessado por nenhum outro objeto.



Modificador de visibilidade

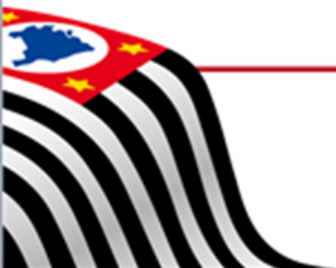
private

A declaração de variáveis de instância com modificador de acesso **private** é conhecida como **ocultamento de dados** ou **ocultamento de informações**.



Modificador de visibilidade

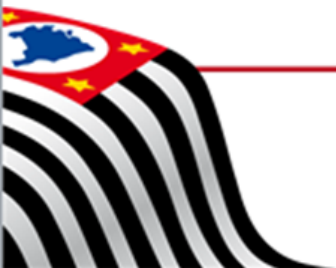
- **protected:** (protegido) o atributo ou método de um objeto dessa classe poderá ser acessado apenas por objetos de classes que sejam derivadas dessa, por meio do mecanismo de herança.



UML

Unified Modeling Language

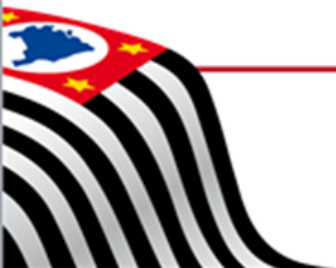
Representação gráfica utilizada para modelar sistemas orientados a objetos.



UML

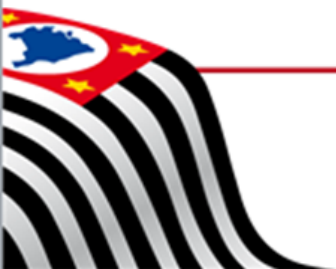
A especificação de uma classe é composta por três regiões:

- o nome da classe
- o conjunto de atributos da classe
- o conjunto de métodos da classe



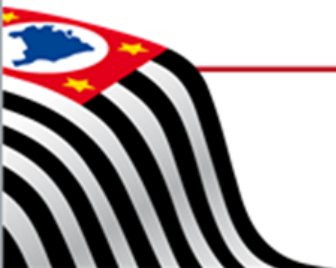
UML

NomeClasse
visibilidade nomeAtributo : tipo = valor default
visibilidade nomeMétodo(listaArgumentos) : tipoRetorno
visibilidade nomeMétodo(listaArgumentos) : tipoRetorno



UML

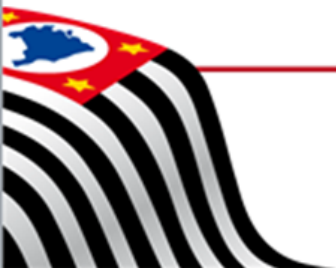
O **nome da classe** é um identificador para a classe, que permite referenciá-la posteriormente.



UML

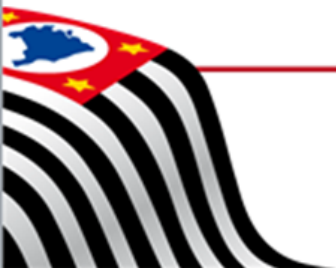
O **conjunto de atributos** descreve as propriedades da classe.

Cada atributo é identificado por um nome e tem um tipo associado.



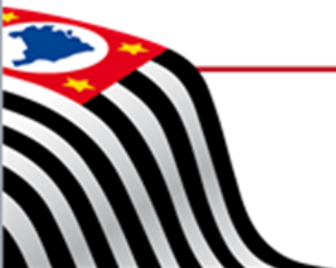
UML

O **atributo** pode ainda ter um valor_default opcional, que especifica um valor inicial para o atributo.



Os tipos primitivos do Java são:

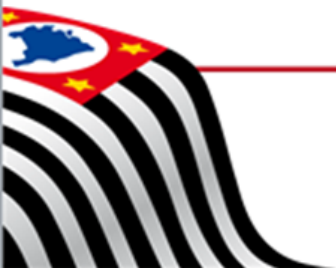
byte, char, short, int, long, float, double e boolean.



UML

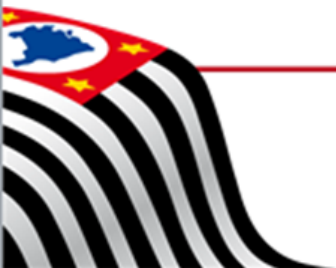
Os **métodos** definem as funcionalidades da classe .

Cada método é especificado por uma assinatura, composta por um identificador para o método, o tipo e sua lista de argumentos.



UML

O **modificador de visibilidade** pode estar presente tanto para atributos como para métodos.

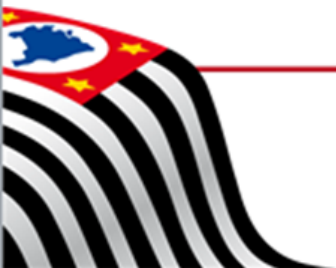


UML

símbolo + modificador público

símbolo - modificador privativo

símbolo # modificador protegido



Exemplo

ContaBancaria
- nomeCorrentista: String - saldo: double
+ depositaValor (valor: double)

